

Discord Scheduled Broadcast Bot - Full Implementation Plan

1. Project Overview

A production-grade Discord automation system that allows server admins to schedule and broadcast messages to selected channels at fixed times or recurring intervals using a dashboard-driven configuration system.

2. Core Features

- Channel selection per server (guild-based configuration)
- Message composer with markdown support
- Fixed-time scheduling (e.g., 9:00 AM UTC+3)
- Interval-based scheduling (every X hours/minutes)
- Enable/disable scheduling jobs
- Logging system for sent/failed messages
- Multi-server support
- Admin-only access control

3. System Architecture

Dashboard (Next.js/V0 UI) → Backend API (Node.js/Express) → Scheduler Engine (BullMQ or node-cron) → Discord Bot (discord.js v14) → Discord Channel Delivery

4. Recommended Tech Stack

Layer	Technology	Purpose
Backend	Node.js + Express.js	API + business logic
Bot	discord.js v14	Discord integration
Scheduler	BullMQ + Redis / node-cron	Job scheduling
Database	PostgreSQL / Supabase	Data storage
Frontend	Next.js / V0	Admin dashboard
Hosting	Railway / Render / Vercel	Deployment

5. Database Schema (Simplified)

Table	Fields
servers	id, guild_id, owner_id
channels	id, guild_id, channel_id
messages	id, content, channel_id
schedules	id, type (cron/interval), time, interval, status
logs	id, schedule_id, status, timestamp

6. API Endpoints

- POST /schedule/create
- POST /schedule/update
- POST /schedule/delete
- GET /channels/:guildId
- GET /logs/:scheduleId

7. Scheduler Design

Two modes supported: Cron-based scheduling for fixed times and interval-based execution using BullMQ queues. Redis handles job persistence and retry mechanisms.

8. Security Considerations

- Role-based access control (admin only)
- Encrypted token storage for Discord bot
- Rate limiting for API endpoints
- Prevention of spam abuse
- Audit logs for all actions

9. Deployment Plan

Backend and bot hosted on Railway/Render. Database on Supabase/PostgreSQL. Redis for queue management. Frontend deployed on Vercel.

10. Development Timeline (3 Days)

- Day 1: Backend + Discord bot setup + database schema
- Day 2: Scheduler engine + API integration
- Day 3: Dashboard + testing + deployment